

MapShop Tanzania - Product Management Integration Guide

Overview

This guide provides complete setup for dynamic product management with categories, full CRUD operations, and product circulation across the system.

✓ What Has Been Implemented

Backend Changes (Django GraphQL)

1. Enhanced CreateProductMutation

- File: `backend/backend/schema/mutations.py`
- Improvements:
 - Better error handling with clear messages
 - Validates user type (SELLER or ADMIN required)
 - Auto-selects seller's first shop if not specified
 - Creates unique product slugs automatically
 - Sets `is_active=True` for newly created products
 - Returns complete product data including category and shop info

2. Existing Mutations

- `UpdateProductMutation` - Already fully implemented
- `DeleteProductMutation` - Sets `is_active=False` (soft delete)
- All mutations require `@login_required` for authentication

3. Backend Queries

- `products` - Returns all active products, filterable by:
 - `category` (slug)
 - `shop_id`
 - `search` (name/description)

- `min_price` / `max_price`
- `categories` - Returns all categories from database
- `myProducts` - Returns seller's products (requires login)

Frontend Changes (Flutter)

1. GraphQL Queries & Mutations

- File: `lib/services/graphql_queries.dart`
- Added:
 - `getCategories` query
 - Enhanced `createProduct` mutation with full response
 - Enhanced `updateProduct` mutation with full response
 - `createShop` mutation for seller shop creation

2. Service Layer

- File: `lib/services/product_service.dart`
 - `getCategories()` - Fetches categories from backend
 - Creates/Updates/Deletes products
 - Fetches seller products
- File: `lib/services/shop_service.dart`
 - `createShop()` - Allows sellers to create shops

3. State Management

- File: `lib/provider/product_provider.dart`
 - Added `categories` list
 - Added `fetchCategories()` method
 - All CRUD methods properly integrated

4. UI Updates

- File: `lib/screens/add_product_screen.dart`
 - Categories loaded dynamically from backend
 - Category dropdown uses Consumer
 - Shows loading state while fetching categories

- Validates category selection before submission
- Better error messages



Product Lifecycle Flow

SELLER WORKFLOW:

- └ Register with user_type='SELLER'
- └ Create Shop (if needed)
 - └ Shop created with seller's location (latitude, longitude)
- └ Add Categories Dynamically
 - └ Categories fetched from: GET /graphql -> categories query
- └ Create Products
 - └ POST /graphql -> createProduct mutation
 - └ Product linked to seller's shop
 - └ is_active=True by default
 - └ immediately visible in system
- └ Manage Products
 - └ Update product details
 - └ Delete/Deactivate products

BUYER WORKFLOW:

- └ Browse Products
 - └ GET /graphql -> products query (is_active=True only)
- └ Filter by Category
 - └ GET /graphql -> products(category="food")
- └ Search by Name
 - └ GET /graphql -> products(search="juice")
- └ View Product Details
 - └ Shows seller name, location, category, price, stock
- └ Place Order
 - └ Creates order in system
 - └ Product circulation complete



Setup Instructions

Backend Setup

1. Ensure Database Has Categories

```
cd backend
python manage.py shell
>>> from apps.products.models import Category
>>> Category.objects.create(
...     name='Food',
...     slug='food',
...     description='Food items',
...     icon='food'
... )
>>> # Repeat for other categories
```

2. Verify GraphQL Endpoint

- Should be accessible at: `http://your-ip:8000/graphql/`
- Test with GraphQL IDE (GraphiQL)

3. Check Seller Shop Requirements

- Sellers MUST create a shop before adding products
- Shop requires: name, description, location (lat/lng), address, phone, email
- Use `createShop` mutation in GraphQL

Frontend Setup

1. Configure GraphQL Endpoint

- File: `lib/services/graphql_config.dart`
- Update IP address in `HttpLink` to match backend server
- Default: `http://192.168.0.213:8000/graphql/`

2. Ensure Authentication

- `AuthLink` automatically adds `Authorization: Bearer {token}` header
- Token stored in `FlutterSecureStorage` with key `access_token`

3. Run Flutter App

```
flutter pub get
flutter run
```



Testing Workflow

Test 1: Categories Loading

1. Open Add Product Screen
2. Verify categories dropdown appears with items
3. If empty, backend categories query may have failed

Test 2: Create Product (Seller)

1. Login as seller
2. Navigate to Add Product
3. Fill form:
 - Name: "juice"
 - Category: Select from dropdown
 - Price: 10000
 - Stock: 1
 - Unit: "kg"
 - Description: "juice mango"
4. Click Add Product
5. Success message should appear
6. Product appears in "My Products"

Test 3: View Product (Buyer)

1. Login as buyer
2. Go to home/browse screen
3. Products should be visible with seller info
4. Filter by category - product should appear
5. Search by "juice" - product should appear

Test 4: Update Product (Seller)

1. Login as seller
2. View seller inventory

3. Edit product details
4. Changes reflected immediately

Test 5: Delete Product (Seller)

1. Login as seller
2. View seller inventory
3. Delete product
4. Product hidden from buyers (`is_active=False`)

Troubleshooting

Problem: "You need to create a shop first"

Solution:

- Seller hasn't created a shop yet
- Go to Shop Settings → Create New Shop
- Fill in shop details with current location
- Then try adding products again

Problem: Categories Dropdown Shows No Items

Solution:

- Check backend has categories: `python manage.py shell > Category.objects.all()`
- Verify GraphQL endpoint is accessible
- Check network tab in DevTools for failed requests
- Clear app cache and restart

Problem: Product Not Appearing After Creation

Solution:

- Product should have `is_active=True`
- Seller should have a shop assigned
- Try refreshing product list (pull-to-refresh)
- Check GraphQL response in backend logs

Problem: "Permission denied" on Create

Solution:

- Check user type is 'SELLER' or 'ADMIN'
- Verify authentication token in secure storage
- Logout and login again to refresh token



Database Schema

```
User (seller)
  ↓
  └─ shops (one-to-many)
      ↓
      └─ products (one-to-many)
          ├── category (many-to-one)
          ├── images (many-to-many)
          └─ ratings (one-to-many)
```

Key Fields:

- `Product.is_active` - Controls visibility to buyers
- `Product.shop_id` - Links to seller's shop
- `Product.category_id` - Links to category
- `Shop.seller_id` - Links to seller user
- `Shop.location` - PostGIS Point (latitude, longitude)



Next Steps (Optional)

1. Add Product Images

- Implement image upload in AddProductScreen
- Store in ProductImage model

2. Add Ratings/Reviews

- Create review system after orders are placed
- Display average rating on product cards

3. Add Product Slugs to Search

- Use slug for URL-friendly product pages
- Improve SEO

4. Implement Wishlist

- Let buyers save favorite products
- Track popular items

5. Add Product Analytics

- Track views, clicks, orders
- Show seller insights



API Reference

GraphQL Mutations

Create Product

```
mutation CreateProduct($categoryId: Int!, $name: String!, ...) {  
  createProduct(categoryId: $categoryId, name: $name, ...) {  
    success  
    message  
    product { id, name, price, category { name } }  
  }  
}
```

Get Categories

```
query {  
  categories {  
    id  
    name  
    slug  
    description  
  }  
}
```

Get Products (Buyers)


```
query GetProducts($search: String, $category: String) {  
  products(search: $search, category: $category) {  
    id, name, price, category { name }, shop { name, address }  
  }  
}
```

Get My Products (Sellers)

```
query {  
  myProducts {  
    id, name, price, stock, category { name }  
  }  
}
```

✓ Implementation Checklist

- ☒ Backend CreateProductMutation improved
- ☒ Frontend categories dynamic loading
- ☒ Product CRUD operations implemented
- ☒ AddProductScreen uses backend categories
- ☒ Products visible to buyers immediately
- ☒ Error messages clear and helpful
- ☒ Shop creation for sellers
- ☒ Product filtering by category/name
- ☒ Authentication integrated

Last Updated: April 14, 2026 **Status:** Ready for testing